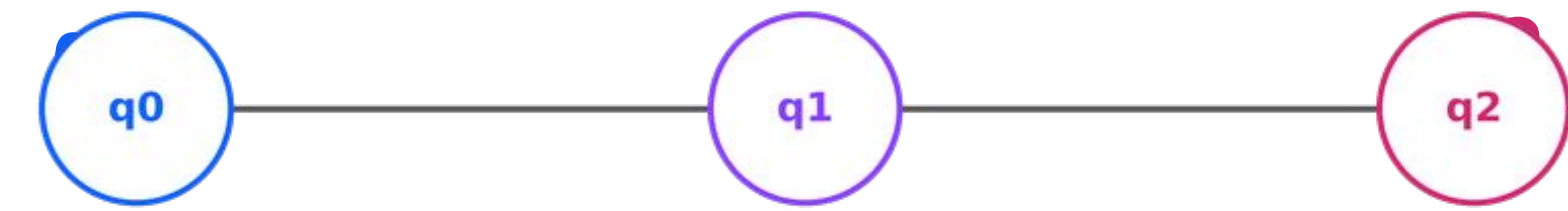


Algorithms, Noise & Mini-Project

From algorithmic idea to
experimental quantum
application



What this talk
prepares you to do

1

Algorithm

Understand the
Deutsch–Jozsa decision
problem and workflow.

3

Hardware

Connect, select a
backend, transpile
and execute.

2

Noise

Model gate errors
and quantify
algorithm
reliability.

4

Application

Build a small
end-to-end
quantum
application.

Part A — Your first quantum algorithm

Deutsch–Jozsa

Use superposition, phase kickback and interference to classify a promised function.

The Deutsch–Jozsa problem

A hidden function

$$f: \{0,1\}^n \rightarrow \{0,1\}$$

Promise: the function is either

- CONSTANT — same output for every input
- BALANCED — 0 and 1 occur equally often

Goal: determine which type the oracle implements.

The algorithm in one picture

Quantum workflow



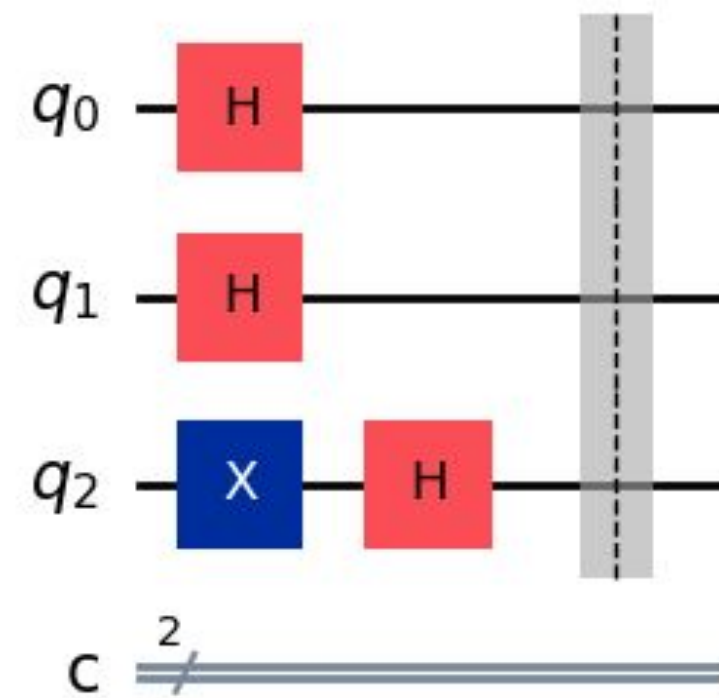
Prepare | Create
superposition |
Encode $f(x)$ as
phase | Interfere |
Measure

The oracle is queried once. The final interference pattern reveals whether the function is constant or balanced.

Registers: input +
ancilla

2 INPUT QUBITS + 1 ANCILLA

Start: $|00\rangle|0\rangle$



The ancilla is prepared as
 $|-\rangle = (|0\rangle - |1\rangle)/\sqrt{2}$.

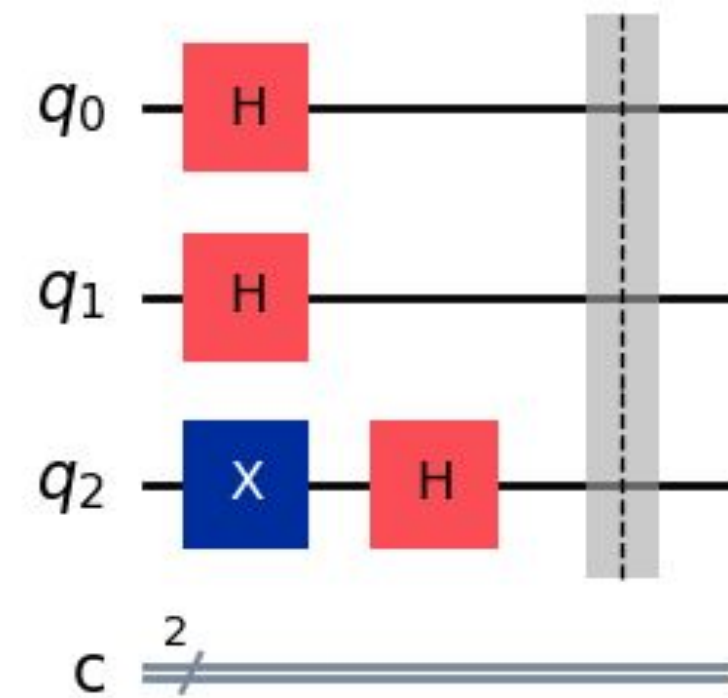
Superposition
creates the search
space

Input register

$$H|0\rangle = |+\rangle = (|0\rangle + |1\rangle)/\sqrt{2}$$

For two inputs:

$$|++\rangle = \frac{1}{2}(|00\rangle + |01\rangle + |10\rangle + |11\rangle)$$



All four inputs are represented in one quantum state.

The oracle does not reveal $f(x)$ by direct measurement.

Instead, it changes the relative phase of amplitudes.

The oracle writes
information into
phase

Phase kickback

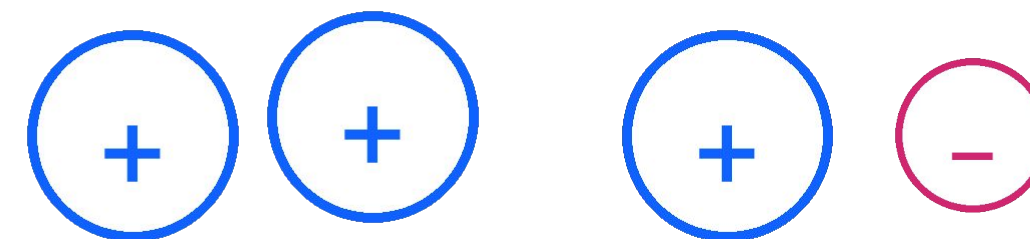
For $f(x_0, x_1) = x_0$:

When $x_0 = 1$, the oracle
applies X to the ancilla.

Because $X|-\rangle = -|-\rangle$, the
input amplitude acquires a
negative phase.

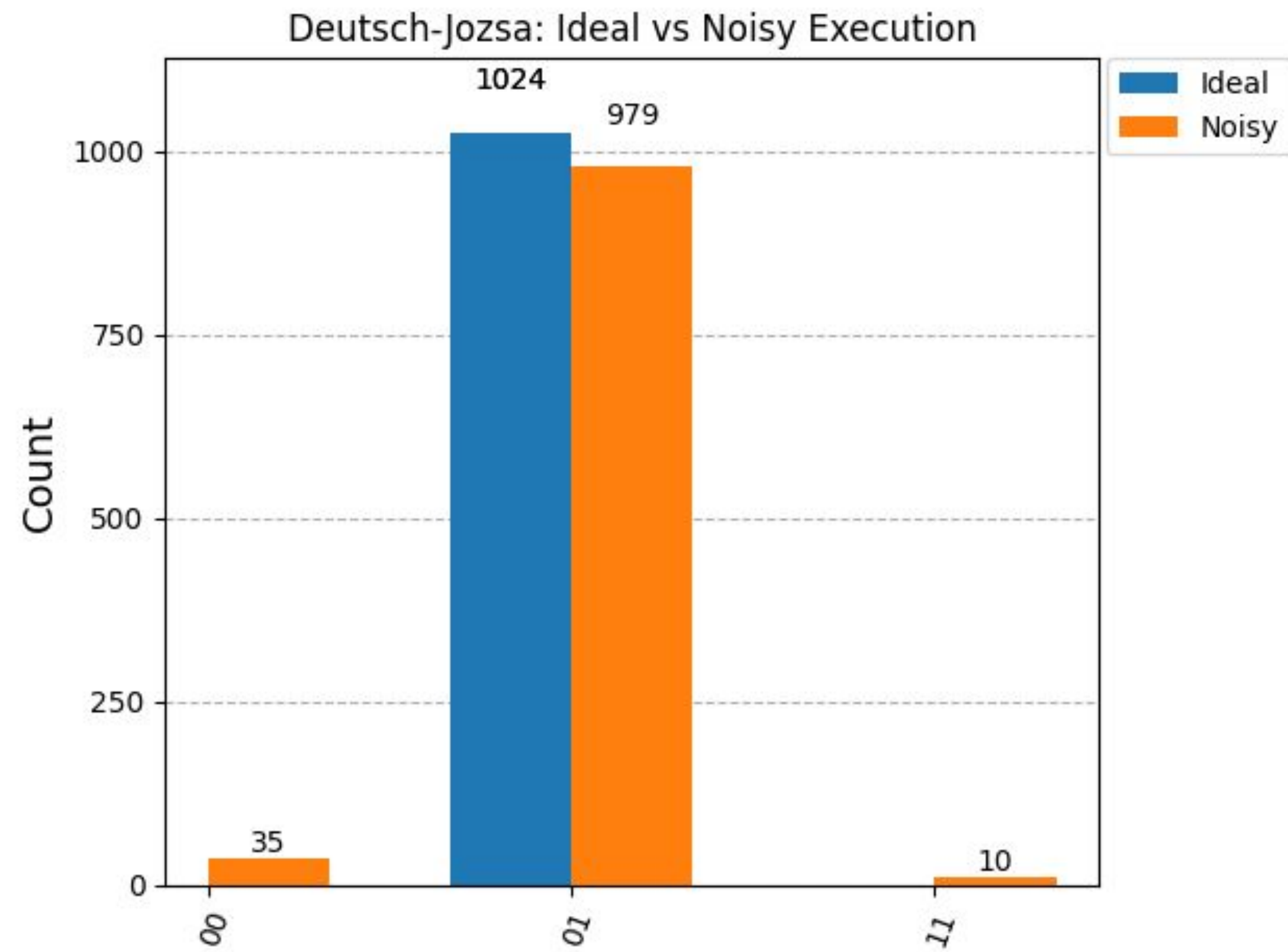
The oracle transforms
amplitude signs, not
measurement probabilities.

That hidden phase is what the
final Hadamard's turn into
measurable interference.



Interference turns phase into an answer

After the final H gates, amplitudes interfere.



Balanced oracle → any non-zero input bitstring

The measurement
becomes the
decision

Constant

Expected input
measurement:

00 → CONSTANT

Expected input measurement:

anything ≠ 00 → BALANCED

Student challenge

Build a different balanced oracle:

$$f(x_0, x_1) = x_0 \oplus x_1$$

Truth table first. Then implement the oracle with two CNOTs.

Key idea

Translate the classical function into controlled operations on the ancilla.

Part B — Noise changes the experiment

Noise & reliability

The algorithm stays the same. The execution environment changes.

Why real quantum
computers are noisy

Gate errors

Physical gates are
imperfect.

Readout

Measurement can
misclassify 0 and 1.

Decoherence

Quantum states interact
with the environment.

Crosstalk

Nearby operations can
influence one another.

A controlled noise model

Depolarizing error

Assign an error probability to gates.

$p_1q = 0.01$ for H and X

$p_2q = 0.03$ for CX

We change the environment — not the algorithm.

Aer lets us reproduce the experiment with a controlled, repeatable noise model.

From counts to reliability

Expected: 01

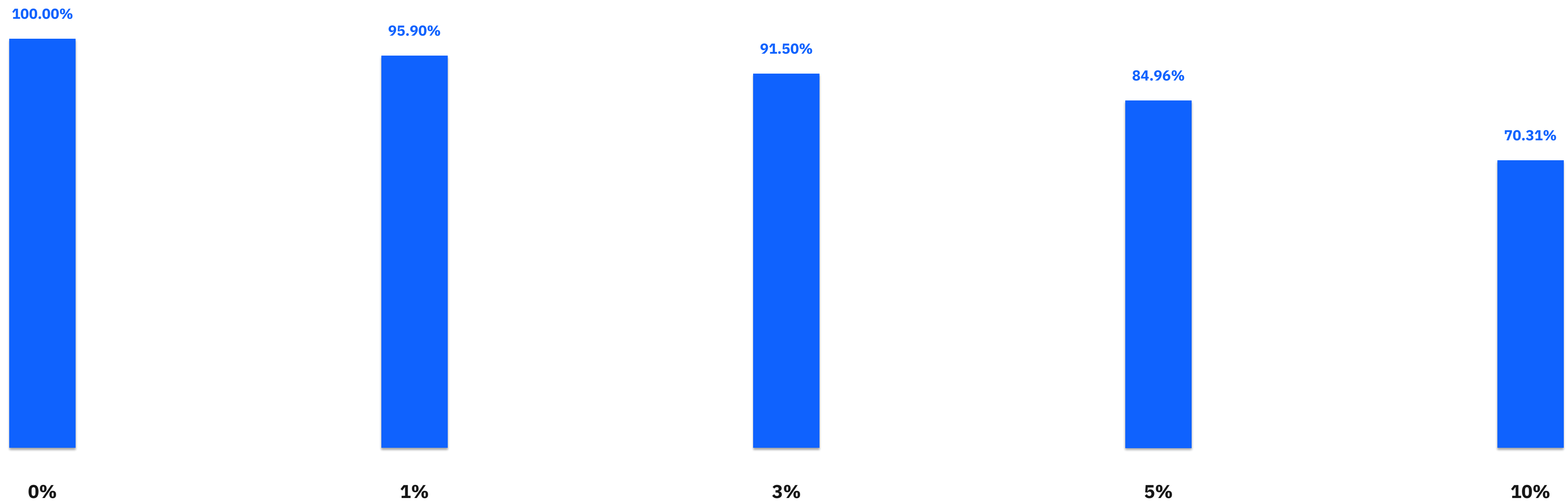
980 correct / 1024 shots

Success $\approx 95.70\%$

Error rate $\approx 4.30\%$

A histogram tells us what happened; the success rate quantifies how reliably the algorithm produced the expected answer.

Success rate



Experiment: increase the noise

Higher simulated gate noise → lower algorithm success rate

Part B — From simulation to quantum hardware

Same algorithm. New execution environment.

1

Ideal

perfect operations

2

Synthetic noise

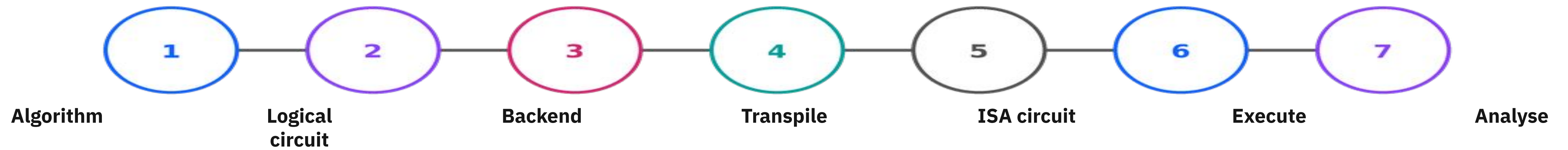
controlled error model

3

Real QPU

physical device

The software-to-hardware workflow



Transpilation adapts the logical circuit to the target processor's constraints.

Transpilation:
logical $\neq$ physical

Logical circuit

transpile

Hardware ISA

3 logical qubits
Depth 5
H, X, CX

156 physical coordinates
Depth 14
RZ, SX, CZ



What the
transpiler must
respect

1

Connectivity

Only physically
connected qubits can
interact directly.

3

Physical qubits

A small logical circuit is
mapped onto selected
hardware qubits.

2

Native gates

Logical gates are
decomposed into
supported instructions.

4

Depth

Extra operations can
increase exposure to
error.

Predict before observing the hardware

Will 01 remain dominant?

Will 00, 10 or 11 appear?

Will hardware success be above or below the synthetic model?

What hardware effects could explain the difference?

A small circuit can still perform well

Real hardware result

997 / 1024

97.36% success

2.64% error

**Hardware $\neq$ ideal — reliability depends on
circuit size, depth and device conditions.**

Three
environments, one
algorithm

Ideal

100.00%



Synthetic 1%

95.90%



Real QPU

97.36%



A simplified $p = 1\%$ model is not a universal hardware error rate.

Part C — Mini-project

Build a quantum random number generator

Turn quantum measurement into an end-to-end application.

Quantum randomness from $|+\rangle$

One qubit

$$|+\rangle = (|0\rangle + |1\rangle)/\sqrt{2}$$

$$P(0) = P(1) = 1/2$$

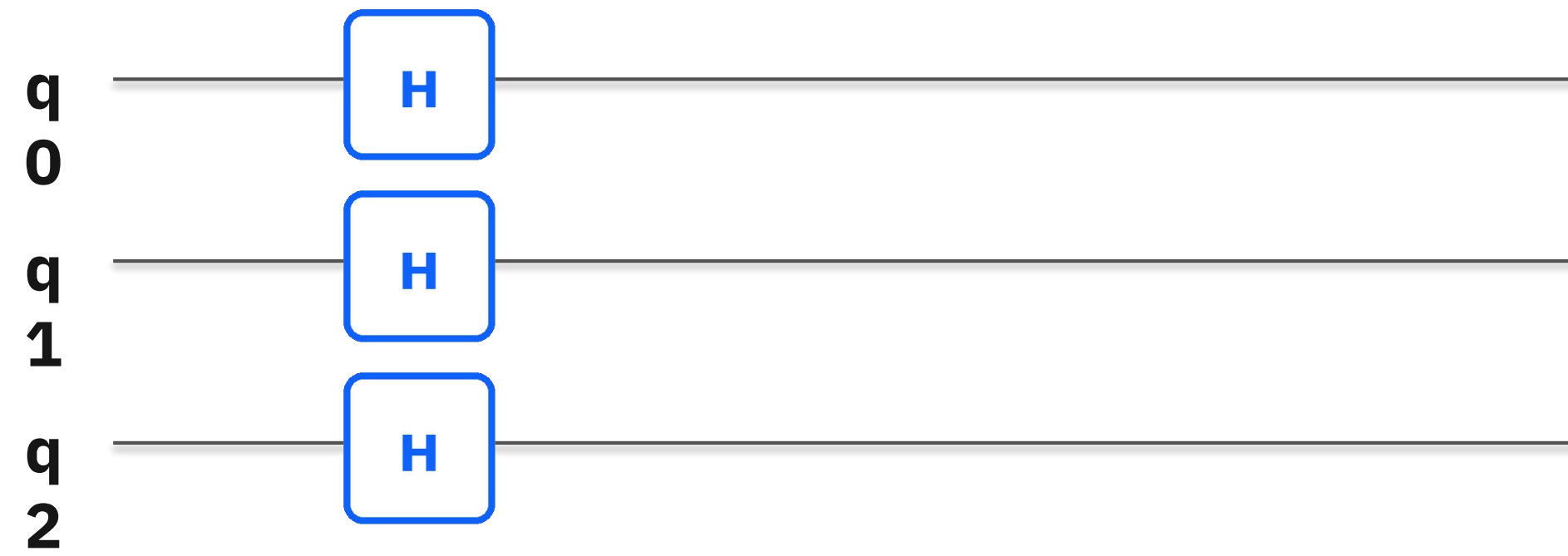
Measurement produces one random bit.

Scale to n qubits $\rightarrow$ n -bit strings $\rightarrow$ integers.

For 3 qubits: 000...111 $\rightarrow$ 0...7.

A 3-qubit QRNG

Circuit



$2^3 = 8$ equally likely bitstrings

Each has theoretical probability $1/8 = 12.5\%$.

Example: $101_2 = 5_{10}$.

Apply H to every qubit, then measure.

From one shot to 1024

shots = 1

one bitstring $\rightarrow$ one integer

shots = 1024

counts $\rightarrow$ probabilities $\rightarrow$ bar chart

Compare each $P(N)$ with 0.125.

Analyze the distribution

Mini-project challenge

Build, execute and analyze a 3-qubit QRNG.

1. Generate 1024 shots
2. Convert bitstrings $\rightarrow$ integers 0...7
3. Compute observed probabilities
4. Plot the distribution
5. Compare with 12.5%
6. Explain sampling variation
7. Optional: run on real hardware

Extension

Modify the design for 4 qubits $\rightarrow$ integers 0...15.

One mini-project
integrates the
course

1

Qubits + gates

Prepare superposition with H.

3

Multi-qubit

Represent n-bit random
values.

2

Measurement

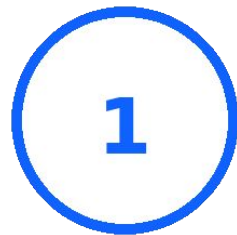
Turn quantum outcomes into
classical data.

4

Noise + hardware

Study how execution
environment affects results.

The complete Qiskit 101 journey



State



Circuit



Algorithm



Noise



Hardware



Application

Algorithm design + execution environment → observed quantum behaviour

Before you open the notebook...

- 1 Can you explain constant vs. balanced?
- 2 Can you describe phase kickback?
- 3 Can you predict the ideal Deutsch–Jozsa output?
- 4 Can you explain why noise creates extra outcomes?
- 5 Can you distinguish a logical circuit from an ISA circuit?
- 6 Can you explain why QRNG probabilities need many shots?

Now build the experiment.

Algorithms $\rightarrow$ noise $\rightarrow$ hardware $\rightarrow$ application